

clusterer_controller Module

OpenSIPS 3.4 — Administration Guide

Zero-config HA clustering via AES-256-GCM encrypted UDP multicast

Author: Yury Kirsanov — VoIPLine Telecom

Date: July 2026

Overview

The `clusterer_controller` module provides automatic peer discovery and topology management for the `clusterer` module via AES-256-GCM encrypted UDP multicast. It eliminates the need for static node configuration or a database — nodes discover each other automatically at startup and the cluster topology is maintained dynamically at runtime.

When `use_controller=1` is set in the `clusterer` module, the `clusterer_controller` module takes over all topology management: it allocates unique node IDs, discovers peer BIN socket addresses, and calls the `clusterer` internal API to add or remove nodes as they join or leave the cluster.

The minimal configuration per node is a single multicast group address. No IP addresses, node IDs, or BIN URLs need to be hardcoded. Any number of nodes can join or leave without any configuration change on the remaining nodes.

Discovery Protocol

All traffic uses UDP multicast to the configured multicast address and port. The cluster group is identified by the combination of both the multicast IP address and the UDP port — two clusters may share the same multicast IP address but use different UDP ports to remain fully isolated from each other. Conversely, clusters on different multicast IPs with the same port are also independent.

Every packet is encrypted and authenticated with AES-256-GCM using a key derived from the configured password. A 12-byte random nonce is generated per packet and a 4-byte timestamp is included in the plaintext to prevent replay attacks (± 30 second window). Packets encrypted with a different password are silently discarded, providing an additional isolation layer between clusters that share a multicast IP.

The following packet types are defined: **ALIVE** — periodic heartbeat sent by every active node every `query_time` seconds.

JOIN_REQ — sent once at startup by a new node, carrying its IP address and BIN socket list.

MEMBER_LIST — sent by the master node in response to a **JOIN_REQ**, carrying the full peer list for election purposes.

NODE_ASSIGN — sent by the master to multicast, allocating a `node_id` and BIN socket record for a specific node. All cluster members receive and apply it.

GOODBYE — sent on graceful shutdown so peers can remove the node immediately without waiting for timeout.

- **ALIVE** — periodic heartbeat sent by every active node every `query_time` seconds.
- **JOIN_REQ** — sent once at startup by a new node, carrying its IP address and BIN socket list.
- **MEMBER_LIST** — sent by the master node in response to a **JOIN_REQ**, carrying the full peer list for election purposes.
- **NODE_ASSIGN** — sent by the master to multicast, allocating a `node_id` and BIN socket record for a specific node. All cluster members receive and apply it.
- **GOODBYE** — sent on graceful shutdown so peers can remove the node immediately without waiting for timeout.

Master Election

The node with the numerically highest IP address is elected master. The election uses a quantized time window so that all NTP-synchronized nodes evaluate the same eligible peer set and always elect the same master deterministically.

Only the master handles JOIN_REQ packets, allocates node_ids, and sends NODE_ASSIGN and MEMBER_LIST packets. Non-master nodes are passive during join events. When the master changes (because a higher-IP node joins or the current master departs), the new master inherits the full peer table it has been building from NODE_ASSIGN packets and can immediately serve new JOIN_REQs.

Dependencies

&osips; Modules

The following modules are required by this module: proto_bin — required so that BIN listeners are registered and available for discovery when clusterer_controller initialises and scans the proto_bin listener list.

clusterer — required with use_controller=1 set, so that the clusterer internal API (load_clusterer_ctrl_binds) is exported and available at initialisation time.

Both dependencies are declared in the module's dep_export_t . OpenSIPS will refuse to start if either dependency is not satisfied. This does not imply that the modules must appear in a particular order in the configuration file — OpenSIPS resolves the dependency at runtime and will initialize the required modules first regardless of loadmodule order.

All other modules that use the clusterer interface (tm , dialog , dispatcher , usrloc etc.) may be loaded in any order relative to clusterer_controller . The clusterer module automatically creates a cluster stub when use_controller=1 is set and a module attempts to register a capability for an unknown cluster.

- proto_bin — required so that BIN listeners are registered and available for discovery when clusterer_controller initialises and scans the proto_bin listener list.
- clusterer — required with use_controller=1 set, so that the clusterer internal API (load_clusterer_ctrl_binds) is exported and available at initialisation time.

External Libraries or Applications

The following libraries must be installed: OpenSSL — for AES-256-GCM encryption and SHA-256 key derivation (libssl/libcrypto). This is already a core OpenSIPS dependency.

- OpenSSL — for AES-256-GCM encryption and SHA-256 key derivation (libssl/libcrypto). This is already a core OpenSIPS dependency.

Exported Parameters

Define a cluster to participate in. The value is a comma-separated key=value string with the following fields: id (required) — positive integer cluster identifier, must match the cluster_id used by clusterer consumers (dialog, usrloc, dispatcher, etc.).

multicast (required) — IPv4 multicast address and UDP port in the form A.B.C.D:PORT . The address must be in the 224.0.0.0/4 range.

password (optional) — AES-256 encryption key material. All nodes in the same cluster must use the same password. Falls back to the global password modparam if not set.

bin_socket (optional) — BIN socket to advertise for this cluster, in the form bin:IP:PORT . Required when multiple clusters are defined. When only one cluster is defined and only one BIN socket exists, the socket is auto-detected from the proto_bin listeners.

This parameter may be set multiple times to participate in multiple clusters simultaneously. Each cluster runs its own independent worker process.

No default value. At least one cluster must be defined.

- id (required) — positive integer cluster identifier, must match the cluster_id used by clusterer consumers (dialog, usrloc, dispatcher, etc.).
- multicast (required) — IPv4 multicast address and UDP port in the form A.B.C.D:PORT . The address must be in the 224.0.0.0/4 range.
- password (optional) — AES-256 encryption key material. All nodes in the same cluster must use the same password. Falls back to the global password modparam if not set.
- bin_socket (optional) — BIN socket to advertise for this cluster, in the form bin:IP:PORT . Required when multiple clusters are defined. When only one cluster is defined and only one BIN socket exists, the socket is auto-detected from the proto_bin listeners.

Explicitly set the local IPv4 address used by the controller for its own node identity and master election. This is the IP that the controller advertises to peers in JOIN_REQ and NODE_ASSIGN packets and uses for the highest-IP master election algorithm.

Note: this parameter controls the controller's identity only. The BIN socket address advertised to clusterer peers is discovered separately from the proto_bin listener list and is independent of this setting. Do not confuse my_ip with the BIN socket IP defined by the socket=bin:IP:PORT core parameter.

When set, the module walks the interface list to find which local interface owns this address and uses that interface for multicast traffic. Startup fails if no local interface owns the given address.

The module supports three identity resolution modes depending on which modparams are provided:

Mode 1 — my_ip set: The given IP is used directly. The owning interface is resolved automatically from the system interface list. Use this mode on multi-homed hosts where you want to pin the controller identity to a specific IP.

Mode 2 — interface set, my_ip not set: The first IPv4 address on the named interface is used as the controller identity IP. A warning is logged if the interface has multiple IPv4 addresses.

Mode 3 — neither set (default): A throw-away UDP socket is connected to the multicast group and getsockname() is called to determine which source IP the kernel would select. The interface name is resolved from the returned IP. Suitable for single-homed hosts.

Default: auto-detected (Mode 3).

Explicitly set the network interface name to use for multicast traffic (e.g. eth0 , enp6s18). The module takes the first IPv4 address assigned to this interface as the controller's identity IP. This

corresponds to Mode 2 described in the `my_ip` parameter documentation above.

Like `my_ip`, this parameter affects the controller's own identity only and has no effect on the BIN socket addresses advertised to clusterer peers.

If the interface has more than one IPv4 address, a warning is logged and the first address (in the order returned by the kernel) is used. Set `my_ip` explicitly to avoid ambiguity on multi-address interfaces.

Ignored if `my_ip` is also set — `my_ip` takes precedence.

Default: auto-detected (Mode 3 — see `my_ip`).

How often (in seconds) each active node sends an ALIVE heartbeat to the multicast group. This value also controls the election window ($3 \times \text{query_time}$) and the peer purge window ($6 \times \text{query_time}$).

Smaller values mean faster failure detection but higher multicast traffic. Valid range: 1–60.

Default value is 5 .

Global default encryption password for all clusters. Used as AES-256 key material (SHA-256 of the password string). All nodes in a cluster must use the same password. Can be overridden per cluster using the `password= key` in the cluster parameter.

Default value is 3eCrEt*5629 . Change this in production.

Exported MI Functions

List all current cluster members with their `node_id`, master/member status and BIN socket addresses. Only peers within the current election window are shown.

Parameters: none

Return full information for a specific node identified by its allocated `node_id`.

Parameters:

`node_id` — the integer `node_id` to look up.

- `node_id` — the integer `node_id` to look up.

Multiple Clusters

A single OpenSIPS instance can participate in multiple clusters simultaneously by repeating the cluster modparam. Each cluster runs an independent worker process with its own multicast socket, peer table, master election, and `node_id` space.

Clusters are distinguished by the combination of their multicast IP address and UDP port. Two useful topologies are possible:

Different ports, same multicast IP — convenient when all clusters share the same L2 segment. The port number alone separates traffic for each cluster. Each cluster's password should also differ to provide an additional encryption barrier.

Different multicast IPs — useful when clusters span different network segments or when multicast routing is scoped differently per cluster.

When multiple clusters are defined and the node has more than one BIN socket, the `bin_socket=` key must be specified in each cluster string to indicate which BIN socket to advertise for that cluster. If only one BIN socket exists, it is used for all clusters automatically.

Each cluster has its own independent clusterer `cluster_id`, allowing different OpenSIPS subsystems to replicate on different clusters:

- Different ports, same multicast IP — convenient when all clusters share the same L2 segment. The port number alone separates traffic for each cluster. Each cluster's password should also differ to provide an additional encryption barrier.
- Different multicast IPs — useful when clusters span different network segments or when multicast routing is scoped differently per cluster.

Example: Multiple clusters — dialog on cluster 1, usrloc on cluster 2

```
# Two BIN sockets, one per cluster
socket=bin:10.0.1.10:5566
socket=bin:10.0.2.10:5566

loadmodule "proto_bin.so"

loadmodule "clusterer.so"
modparam("clusterer", "use_controller", 1)

loadmodule "clusterer_controller.so"
# Cluster 1 – dialog replication group, LAN segment 10.0.1.0/24
modparam("clusterer_controller", "cluster",
    "id=1,multicast=239.0.90.1:3333,bin_socket=bin:10.0.1.10:5566")
# Cluster 2 – usrloc replication group, LAN segment 10.0.2.0/24
modparam("clusterer_controller", "cluster",
    "id=2,multicast=239.0.90.1:3334,bin_socket=bin:10.0.2.10:5566")

loadmodule "dialog.so"
modparam("dialog", "dialog_replication_cluster", 1)

loadmodule "usrloc.so"
modparam("usrloc", "cluster_id", 2)
```

Example: Multiple clusters — same multicast IP, different ports

```
socket=bin:10.22.23.191:3857

loadmodule "proto_bin.so"

loadmodule "clusterer.so"
modparam("clusterer", "use_controller", 1)

loadmodule "clusterer_controller.so"
modparam("clusterer_controller", "cluster",
    "id=1,multicast=239.0.90.1:3333,password=ClusterOneSecret")
modparam("clusterer_controller", "cluster",
    "id=2,multicast=239.0.90.1:3334,password=ClusterTwoSecret")

loadmodule "dialog.so"
modparam("dialog", "dialog_replication_cluster", 1)

loadmodule "dispatcher.so"
modparam("dispatcher", "cluster_id", 2)
```

Configuration Example

The following example shows a minimal two-module configuration for zero-config HA clustering with dialog replication. The loadmodule order does not matter — the dependency system enforces correct initialization order automatically.

The same configuration file (with the node-specific `socket=bin:IP:PORT` line changed per node) is used on every node. No other per-node customization is required.

Example: Minimal HA cluster configuration

```
# Each node needs an explicit BIN socket (no wildcard)
socket=bin:10.22.23.191:3857

loadmodule "proto_bin.so"

loadmodule "clusterer.so"
modparam("clusterer", "use_controller", 1)
modparam("clusterer", "sharing_tag", "vip1/1=active")
modparam("clusterer", "ping_interval", 4)
modparam("clusterer", "ping_timeout", 1500)

loadmodule "clusterer_controller.so"
modparam("clusterer_controller", "cluster",
■"id=1,multicast=239.0.90.1:3333")
modparam("clusterer_controller", "password", "MyStrongPassw0rd!")

loadmodule "tm.so"
modparam("tm", "tm_replication_cluster", 1)

loadmodule "dialog.so"
modparam("dialog", "dialog_replication_cluster", 1)
modparam("dialog", "cluster_auto_sync", 1)

loadmodule "dispatcher.so"
modparam("dispatcher", "cluster_id", 1)
modparam("dispatcher", "cluster_probing_mode", "distributed")
```

Limitations

IPv4 only. IPv6 multicast is not currently supported.

Wildcard BIN sockets (`bin*:PORT`) are rejected at startup. An explicit IP address must be used in the `socket=` line.

Node IDs are not persistent across a full cluster restart (all nodes down simultaneously). IDs are reallocated starting from 1 when the cluster reforms. This has no operational impact as long as at least one node remains up during rolling restarts.

The replay protection window requires NTP synchronisation between all nodes. The maximum accepted clock skew is 30 seconds.

The multicast network must support IP multicast routing between all cluster nodes. Nodes on different L3 segments require PIM or similar multicast routing.

- IPv4 only. IPv6 multicast is not currently supported.
- Wildcard BIN sockets (bin:*:PORT) are rejected at startup. An explicit IP address must be used in the socket= line.
- Node IDs are not persistent across a full cluster restart (all nodes down simultaneously). IDs are reallocated starting from 1 when the cluster reforms. This has no operational impact as long as at least one node remains up during rolling restarts.
- The replay protection window requires NTP synchronisation between all nodes. The maximum accepted clock skew is 30 seconds.
- The multicast network must support IP multicast routing between all cluster nodes. Nodes on different L3 segments require PIM or similar multicast routing.